

Huttas.com to CRM Website Request Webhook

Client Developer Integration Guide

Secure server-to-server connection for the existing Contact Us form.

PRODUCTION ENDPOINT

POST [https://hutta-home-services-dashboard-main.onrender.com/
api/integrations/website-requests](https://hutta-home-services-dashboard-main.onrender.com/api/integrations/website-requests)

The shared secret is provided separately through a secure channel. Never place it in the PDF, frontend code, or source control.

1. Customer submits form

2. Website server signs request

3. CRM returns REQ reference

Document purpose: Instructions for the huttas.com developer to send the existing Contact Us form securely into the Hutta Home Services CRM.

CRM endpoint:

```
POST https://hutta-home-services-dashboard-main.onrender.com/api/integrations/website-requests
```

The website must send this request from its server. Do not place the shared secret or signing code in browser JavaScript.

1. Information the Hutta CRM administrator will provide

Item	Value
Production endpoint	https://hutta-home-services-dashboard-main.onrender.com/api/integrations/website-requests
HTTP method	POST
Content type	application/json
Signing method	HMAC-SHA256
Shared secret	Provided separately through a secure password manager or encrypted message
Timestamp header	X-Huttas-Timestamp
Signature header	X-Huttas-Signature

The shared secret must be stored as a server-side environment variable named `HUTTAS_WEBHOOK_SECRET`. Do not send it by ordinary email and do not include it in screenshots, source control, frontend bundles, or logs.

2. Existing form field mapping

Website form field	JSON field	Required	Notes
Name	<code>name</code>	Yes	Maximum 160 characters
Phone	<code>phone</code>	Yes	Must contain 10 to 15 digits
Email	<code>email</code>	Yes	Valid customer email address
Service Details	<code>serviceDetails</code>	No	Maximum 5,000 characters
Marketing SMS checkbox	<code>marketingSmsConsent</code>	No	Send a JSON Boolean: <code>true</code> or <code>false</code>
Generated by website	<code>externalSubmissionId</code>	Yes	Stable UUID/idempotency key, 8 to 128 allowed characters
Generated by website	<code>submittedAt</code>	Yes	ISO-8601 UTC timestamp

Example JSON body:

```
{
  "externalSubmissionId": "f82f949d-90f1-46f5-8178-731f5b19661e",
  "submittedAt": "2026-07-15T11:06:00.000Z",
  "name": "Jane Customer",
  "phone": "4801234567",
  "email": "jane@example.com",
  "serviceDetails": "I want my landscaping done.",
  "marketingSmsConsent": false
}
```

3. Signing algorithm

1. Build the final JSON body exactly once and retain its exact UTF-8 bytes.
2. Generate the current Unix timestamp in seconds, for example **1784113560**.
3. Build the signed message as **timestamp + "." + rawJsonBody**.
4. Compute the lowercase hexadecimal HMAC-SHA256 digest using the shared secret.
5. Send the timestamp and digest in the required headers.

Formula:

```
signature = lowercase_hex(
  HMAC_SHA256(shared_secret, timestamp + "." + exact_raw_json_body)
)
```

Required headers:

```
Content-Type: application/json
X-Huttas-Timestamp: 1784113560
X-Huttas-Signature: <64 lowercase hexadecimal characters>
```

Important: sign the exact body that is transmitted. Re-serializing or changing whitespace after signing will invalidate the signature. The CRM rejects timestamps more than five minutes old.

4. Node.js example

```
const crypto = require('crypto');

async function sendHuttasRequest(form) {
  const submission = {
    externalSubmissionId: crypto.randomUUID(),
    submittedAt: new Date().toISOString(),
    name: form.name.trim(),
    phone: form.phone.trim(),
    email: form.email.trim(),
    serviceDetails: (form.serviceDetails || '').trim(),
    marketingSmsConsent: form.marketingSmsConsent === true
  };

  const rawBody = JSON.stringify(submission);
  const timestamp = Math.floor(Date.now() / 1000).toString();
  const signature = crypto
    .createHmac('sha256', process.env.HUTTAS_WEBHOOK_SECRET)
    .update(`${timestamp}.${rawBody}`, 'utf8')
    .digest('hex');

  const response = await fetch(
    'https://hutta-home-services-dashboard-main.onrender.com' +
    '/api/integrations/website-requests',
    {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        'X-Huttas-Timestamp': timestamp,
        'X-Huttas-Signature': signature
      },
      body: rawBody,
      signal: AbortSignal.timeout(10000)
    }
  );

  const result = await response.json();
  if (!response.ok) {
    throw new Error(result.message || `CRM returned HTTP ${response.status}`);
  }
  return result;
}
```

Generate `externalSubmissionId` once when the customer submits. If the CRM call times out, retry using the same submission object and the same external ID, but generate a fresh timestamp and signature.

5. PHP / WordPress example

This function can be called from the server-side handler or plugin hook that receives the website form. The exact hook depends on the WordPress form plugin.

```

function huttas_uuid_v4() {
    $data = random_bytes(16);
    $data[6] = chr((ord($data[6]) & 0x0f) | 0x40);
    $data[8] = chr((ord($data[8]) & 0x3f) | 0x80);
    return vsprintf('%s%s-%s-%s-%s-%s', str_split(bin2hex($data), 4));
}

function send_huttas_crm_request($form) {
    $submission = array(
        'externalSubmissionId' => huttas_uuid_v4(),
        'submittedAt' => gmdate('c'),
        'name' => trim($form['name']),
        'phone' => trim($form['phone']),
        'email' => trim($form['email']),
        'serviceDetails' => trim($form['serviceDetails'] ?? ''),
        'marketingSmsConsent' => !empty($form['marketingSmsConsent'])
    );

    $raw_body = wp_json_encode($submission);
    $timestamp = strval(time());
    $secret = getenv('HUTTAS_WEBHOOK_SECRET');
    $signature = hash_hmac(
        'sha256', $timestamp . '.' . $raw_body, $secret
    );

    $response = wp_remote_post(
        'https://hutta-home-services-dashboard-main.onrender.com' .
        '/api/integrations/website-requests',
        array(
            'timeout' => 10,
            'headers' => array(
                'Content-Type' => 'application/json',
                'X-Huttas-Timestamp' => $timestamp,
                'X-Huttas-Signature' => $signature
            ),
            'body' => $raw_body
        )
    );

    if (is_wp_error($response)) {
        throw new Exception($response->get_error_message());
    }
    $status = wp_remote_retrieve_response_code($response);
    $result = json_decode(wp_remote_retrieve_body($response), true);
    if ($status < 200 || $status >= 300) {
        throw new Exception($result['message'] ?? 'CRM request failed');
    }
    return $result;
}

```

For reliable retries, save the generated submission or at least its `externalSubmissionId` before the first CRM request. Do not create a new UUID for a retry.

6. Success and error handling

New submission response (HTTP 201):

```
{
  "success": true,
  "requestReference": "REQ-2026-000123",
  "status": "received",
  "duplicate": false
}
```

Safe retry/duplicate response (HTTP 200) returns the same reference with `duplicate: true`.

HTTP status	Meaning	Website action
200	Previously accepted duplicate	Show success using the returned reference
201	New request accepted	Show success using the returned reference
400	Missing or invalid field/body	Log safely and show a generic submission error
401	Missing, invalid, or expired signature	Check secret, server clock, body bytes, and headers
413	Body exceeds 32 KB	Reduce submitted content
429	Too many requests	Retry later using the same external ID
500	CRM processing failure	Retry later using the same external ID

Recommended customer success message:

```
Thank you. We received your request.
Your reference number is REQ-2026-000123.
Our team will contact you.
```

Do not display success until the CRM returns HTTP 200 or 201. Never display the signature, secret, raw CRM error, internal ID, or stack trace to the customer.

7. Retry and idempotency rules

- Use a 10-second request timeout.
- Retry network timeouts, HTTP 429, and HTTP 500 responses.
- Suggested delays: 2 seconds, 10 seconds, then 60 seconds.
- Always reuse the original `externalSubmissionId`.
- Rebuild the timestamp and signature for each retry.
- Do not retry HTTP 400 until the submitted data is corrected.
- For HTTP 401, verify configuration rather than repeatedly retrying.
- Store the returned `requestReference` with the website submission if the platform supports it.

The CRM's external submission ID prevents a timeout retry from creating a second Order.

8. Security requirements

- Send only through HTTPS.
- Keep `HUTTAS_WEBHOOK_SECRET` on the website server.
- Never include the secret in HTML, frontend JavaScript, analytics, Git, screenshots, or normal logs.
- Redact customer name, phone, email, body, signature, and secret from error logs where possible.
- Ensure the website server clock is synchronized automatically.
- Restrict who can read or change the production environment variable.
- Rotate the secret only in coordination with the Hutta CRM administrator; both systems must use the same value.
- Keep marketing SMS consent as a Boolean and do not treat it as permission for unrelated marketing channels.

9. Acceptance test

The website developer and Hutta CRM administrator should perform this test together:

1. Confirm the same shared secret is configured on both servers.
2. Submit the public form with an obvious test customer name.
3. Confirm the website shows `REQ- . . .` from the webhook response.
4. Confirm exactly one Order appears in CRM Orders.
5. Confirm it appears in Workflow Center > Request Received.
6. Confirm the Order shows Website, Request Received, and Unquoted.
7. Confirm no Payment was created.
8. Confirm the customer receipt email arrives.
9. Confirm the operations email and in-app notification arrive.
10. Resend the exact test submission with the same external ID and confirm no duplicate Order is created.
11. Delete or clearly close the test Order after approval.

10. Client completion checklist

- [] Server-side form hook identified.
- [] Shared secret stored securely as `HUTTAS_WEBHOOK_SECRET`.
- [] Stable UUID generated once per customer submission.
- [] Exact raw JSON body signed with HMAC-SHA256.
- [] Timestamp and signature headers added.
- [] Existing form fields mapped correctly.
- [] HTTP 200 and 201 handled as success.
- [] `requestReference` shown to the customer.
- [] Safe retries reuse the same external ID.
- [] Secrets and customer data excluded from logs.
- [] Joint acceptance test completed.

Support handoff

When reporting an integration issue, provide only:

- Date and time of the attempt, including timezone.
- HTTP status code.
- `externalSubmissionId`.
- Returned `requestReference`, if present.
- A short error category.

Do not send the shared secret, signature, or full customer payload in ordinary email or chat.